

Chapter 11 – Strings and Characters

Outline

- 11.1 Introduction**
- 11.2 Fundamentals of Characters and Strings**
- 11.3 Class String**
 - 11.3.1 String Constructors**
 - 11.3.2 String Methods length, charAt and getChars**
 - 11.3.3 Comparing Strings**
 - 11.3.4 Locating Characters and Substrings in Strings**
 - 11.3.5 Extracting Substrings from Strings**
 - 11.3.6 Concatenating Strings**
 - 11.3.7 Miscellaneous String Methods**
 - 11.3.8 String Method valueOf**
- 11.4 Class StringBuffer**
 - 11.4.1 StringBuffer Constructors**
 - 11.4.2 StringBuffer Methods length, capacity, setLength and ensureCapacity**
 - 11.4.3 StringBuffer Methods charAt, setCharAt, getChars and reverse**



Chapter 11 – Strings and Characters

11.4.4 StringBuffer append Methods

11.4.5 StringBuffer Insertion and Deletion Methods

11.5 Class Character

11.6 Class StringTokenizer



11.1 Introduction

- String and character processing
 - Class `java.lang.String`
 - Class `java.lang.StringBuffer`
 - Class `java.lang.Character`
 - Class `java.util.StringTokenizer`



11.2 Fundamentals of Characters and Strings

- Characters
 - “Building blocks” of non-numeric data
 - 'a', '\$', '4'
- String
 - Sequence of characters treated as single unit
 - May include letters, digits, etc.
 - Object of class `String`
 - `String name = “Frank N. Stein”;`



11.3.1 String Constructors

- Class `String`
 - Provides nine constructors
 - Null constructor `String()` has no characters and a length of zero
 - `String (array, offset, number of characters)`



```

1  // Fig. 11.1: StringConstructors.java
2  // String class constructors.
3  import javax.swing.*;
4
5  public class StringConstructors {
6
7      public static void main( String args[] )
8      {
9          char charArray[] = { 'b', 'i', 'r', 't', 'h', ' ', 'd', 'a', 'y' };
10         byte byteArray[] = { ( byte ) 'n', ( byte ) 'e',
11                             ( byte ) 'w', ( byte ) ' ', ( byte ) 'y',
12                             ( byte ) 'e', ( byte ) 'a', ( byte ) 'r' };
13
14         String s = new String( "hello" );
15
16         // use String constructors
17         String s1 = new String( );
18         String s2 = new String( s );
19         String s3 = new String( charArray );
20         String s4 = new String( charArray, 6, 3 );
21         String s5 = new String( byteArray, 4, 4 );
22         String s6 = new String( byteArray );

```

String default constructor
instantiates *empty string*

Constructor copies String

Constructor copies character array

Constructor copies
character-array subset

Constructor copies byte-array
subset

Constructor copies byte array

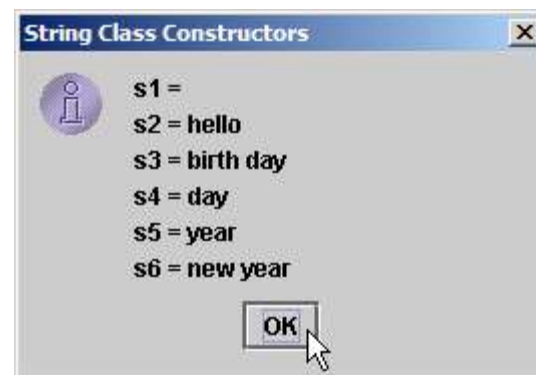


Outline



StringConstruct
ors.java

```
23
24 // append strings to output
25 String output = "s1 = " + s1 + "\ns2 = " + s2 + "\ns3 = " + s3 +
26 "\ns4 = " + s4 + "\ns5 = " + s5 + "\ns6 = " + s6;
27
28 JOptionPane.showMessageDialog( null, output,
29 "String Class Constructors", JOptionPane.INFORMATION_MESSAGE );
30
31 System.exit( 0 );
32 }
33
34 } // end class StringConstructors
```



11.3.2 String Methods Length, charAt and getChars

- Method `length`
 - Determine `String` length
 - Like arrays, `Strings` always “know” their size
 - Unlike array, `Strings` do not have length instance variable
 - `s1.length()`
- Method `charAt`
 - Get character at specific location in `String`
 - `s1.charAt( offset )`
- Method `getChars`
 - Get entire set of characters in `String`
 - `s1.getChars( start, first after, charArray, start );`





Outline



StringMiscellane
ous.java

Line 16

Line 21

```
1 // Fig. 11.2: StringMiscellaneous.java
2 // This program demonstrates the length, charAt and getChars
3 // methods of the String class.
4 import javax.swing.*;
5
6 public class StringMiscellaneous {
7
8     public static void main( String args[] )
9     {
10         String s1 = "hello there";
11         char charArray[] = new char[ 5 ];
12
13         String output = "s1: " + s1;
14
15         // test length method
16         output += "\nLength of s1: " + s1.length();
17
18         // loop through characters in s1 and display reversed
19         output += "\nThe string reversed is: ";
20
21         for ( int count = s1.length() - 1; count >= 0; count-- )
22             output += s1.charAt( count ) + " ";
```

Determine number of
characters in **String s1**

Append **s1's** characters
in reverse order to
String output

Outline

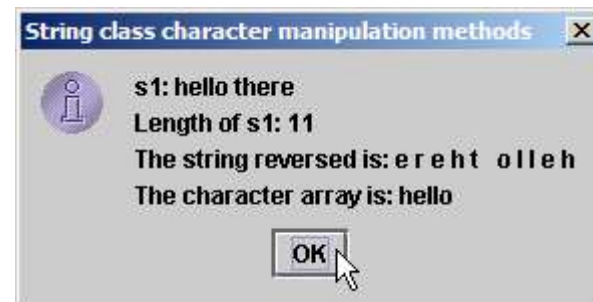
StringMiscellaneous.java

Line 25



Copy (some of) s1's
characters to charArray

```
23 // copy characters from string into charArray
24 s1.getChars( 0, 5, charArray, 0 );
25 output += "\nThe character array is: ";
26
27
28 for ( int count = 0; count < charArray.length; count++ )
29     output += charArray[ count ];
30
31 JOptionPane.showMessageDialog( null, output,
32     "String class character manipulation methods",
33     JOptionPane.INFORMATION_MESSAGE );
34
35 System.exit( 0 );
36 }
37
38 } // end class StringMiscellaneous
```



11.3.3 Comparing Strings

- Comparing `String` objects
 - Should not use `==` (true only if the strings are at the same address, i.e., same string)
 - Primitives contain values, objects contain addresses
 - Method `equals` (true if the strings are identical)
 - Method `equalsIgnoreCase`
 - Method `compareTo`
 - `a.compareTo(b)`, 0 if a and b are same, negative if $a < b$, positive if $a > b$
 - Method `regionMatches`
 - `a.regionMatches(start, b, start, num of chars)` (true if the strings are identical)





Outline

12

StringCompare.j
ava

Line 18

Line 24

```
1  // Fig. 11.3: StringCompare.java
2  // String methods equals, equalsIgnoreCase, compareTo and regionMatches.
3  import javax.swing.JOptionPane;
4
5  public class StringCompare {
6
7      public static void main( String args[] )
8      {
9          String s1 = new String( "hello" ); // s1 is a copy of "hello"
10         String s2 = "goodbye";
11         String s3 = "Happy Birthday";
12         String s4 = "happy birthday";
13
14         String output = "s1 = " + s1 + "\ns2 = " + s2 + "\ns3 = " + s3 +
15             "\ns4 = " + s4 + "\n\n";
16
17         // test for equality
18         if ( s1.equals( "hello" ) ) ← // true
19             output += "s1 equals \"hello\"\n";
20         else
21             output += "s1 does not equal \"hello\"\n";
22
23         // test for equality with ==
24         if ( s1 == "hello" ) ← // false; they are not
25             output += "s1 equals \"hello\"\n";
26         else
27             output += "s1 does not equal \"hello\"\n";
```

Method `equals` tests two
objects for equality using
lexicographical comparison

Equality operator (`==`) tests
if both references refer to
same object in memory



Outline



StringCompare.java

Lines 36 to 40

Line 43 and 49

```

28
29 // test for equality (ignore case)
30 if ( s3.equalsIgnoreCase( s4 ) ) // true
31     output += "s3 equals s4\n";
32 else
33     output += "s3 does not equal s4\n";
34
35 // test compareTo
36 output += "\ns1.compareTo( s2 ) is " + s1.compareTo( s2 ) +
37     "\ns2.compareTo( s1 ) is " + s2.compareTo( s1 ) +
38     "\ns1.compareTo( s1 ) is " + s1.compareTo( s1 ) +
39     "\ns3.compareTo( s4 ) is " + s3.compareTo( s4 ) +
40     "\ns4.compareTo( s3 ) is " + s4.compareTo( s3 ) + "\n\n";
41
42 // test regionMatches (case sensitive)
43 if ( s3.regionMatches( 0, s4, 0, 5 ) )
44     output += "First 5 characters of s3 and s4 match\n";
45 else
46     output += "First 5 characters of s3 and s4 do not match\n";
47
48 // test regionMatches (ignore case)
49 if ( s3.regionMatches( true, 0, s4, 0, 5 ) )
50     output += "First 5 characters of s3 and s4 match";
51 else
52     output += "First 5 characters of s3 and s4 do not match";

```

Test two objects for equality, but ignore case of letters in Strings

Method compareTo compares String objects

Method regionMatches compares portions of two String objects for equality

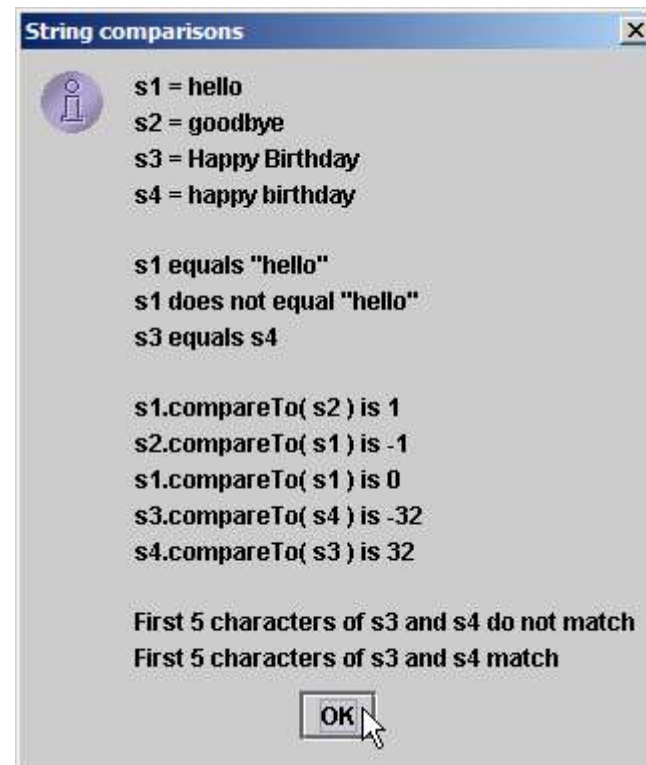


Outline

14

StringCompare.j
ava

```
53
54     JOptionPane.showMessageDialog( null, output,
55         "String comparisons", JOptionPane.INFORMATION_MESSAGE );
56
57     system.exit( 0 );
58 }
59
60 } // end class StringCompare
```





Outline

15

StringStartEnd.
java

Line 15

Line 24

```
1 // Fig. 11.4: StringStartEnd.java
2 // String methods startsWith and endsWith.
3 import javax.swing.*;
4
5 public class StringStartEnd {
6
7     public static void main( String args[] )
8     {
9         String strings[] = { "started", "starting", "ended", "ending" };
10        String output = "";
11
12        // test method startsWith
13        for ( int count = 0; count < strings.length; count++ )
14
15            if ( strings[ count ].startsWith( "st" ) )
16                output += "\"" + strings[ count ] + "\" starts with \"st\"\\n";
17
18        output += "\\n";
19
20        // test method startsWith starting from position
21        // 2 of the string
22        for ( int count = 0; count < strings.length; count++ )
23
24            if ( strings[ count ].startsWith( "art", 2 ) )
25                output += "\"" + strings[ count ] +
26                "\" starts with \"art\" at position 2\\n";
```

Method **startsWith**
determines if **String** starts
with specified characters



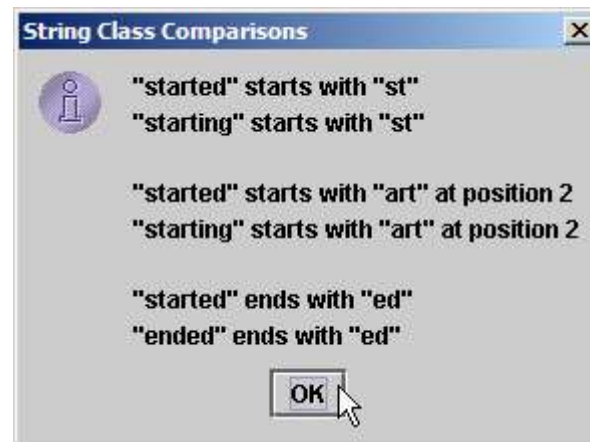
Outline

StringStartEnd.
java

Line 33

```
27
28     output += "\n";
29
30     // test method endsWith
31     for ( int count = 0; count < strings.length; count++ )
32
33         if ( strings[ count ].endsWith( "ed" ) )
34             output += "\"\" + strings[ count ] + "\" ends with \"ed\\n\"";
35
36     JOptionPane.showMessageDialog( null, output,
37         "String Class Comparisons", JOptionPane.INFORMATION_MESSAGE );
38
39     System.exit( 0 );
40 }
41
42 } // end class StringStartEnd
```

Method endsWith
determines if String ends
with specified characters



11.3.4 Locating Characters and Substrings in Strings

- Search for characters in `String`
 - Method `indexOf`
 - `indexOf(char)`, `indexOf(char, start)`
 - `indexOf(string)`, `indexOf(string, start)`
 - Method `lastIndexOf`
 - `lastIndexOf(char)`, `lastIndexOf(char, start)`
 - `lastIndexOf(string)`,
`lastIndexOf(string, start)`





Outline



StringIndexMeth
ods.java

Lines 12-16

Lines 19-26

```

1  // Fig. 11.5: StringIndexMethods.java
2  // String searching methods indexOf and lastIndexOf.
3  import javax.swing.*;
4
5  public class StringIndexMethods {
6
7      public static void main( String args[] )
8      {
9          String letters = "abcdefghijklmabcdefghijklm";
10
11         // test indexOf to locate a character in a string
12         String output = "'c' is located at index " + letters.indexOf( 'c' );
13
14         output += "\n'a' is located at index " + letters.indexOf( 'a' );
15
16         output += "\n'$' is located at index " + letters.indexOf( '$' );
17
18         // test lastIndexOf to find a character in a string
19         output += "\n\nLast 'c' is located at index " +
20             letters.lastIndexOf( 'c' );
21
22         output += "\n\nLast 'a' is located at index " +
23             letters.lastIndexOf( 'a', 25 );
24
25         output += "\n\nLast '$' is located at index " +
26             letters.lastIndexOf( '$' );
27

```

Method `indexOf` finds first occurrence of character in String

Method `lastIndexOf` finds last occurrence of character in String



Outline

StringIndexMethods.java

Lines 29-46

Methods `indexOf` and `lastIndexOf` can also find occurrences of substrings

```

28 // test indexOf to locate a substring in a string
29 output += "\n\n\"def\" is located at index " +
30     letters.indexOf( "def" );
31
32 output += "\n\n\"def\" is located at index " +
33     letters.indexOf( "def", 7 );
34
35 output += "\n\n\"hello\" is located at index " +
36     letters.indexOf( "hello" );
37
38 // test lastIndexOf to find a substring in a string
39 output += "\n\nLast \"def\" is located at index " +
40     letters.lastIndexOf( "def" );
41
42 output += "\n\nLast \"def\" is located at index " +
43     letters.lastIndexOf( "def", 25 );
44
45 output += "\n\nLast \"hello\" is located at index " +
46     letters.lastIndexOf( "hello" );
47
48 JOptionPane.showMessageDialog( null, output,
49     "String searching methods", JOptionPane.INFORMATION_MESSAGE );
50
51 System.exit( 0 );
52 }
53
54 } // end class StringIndexMethods

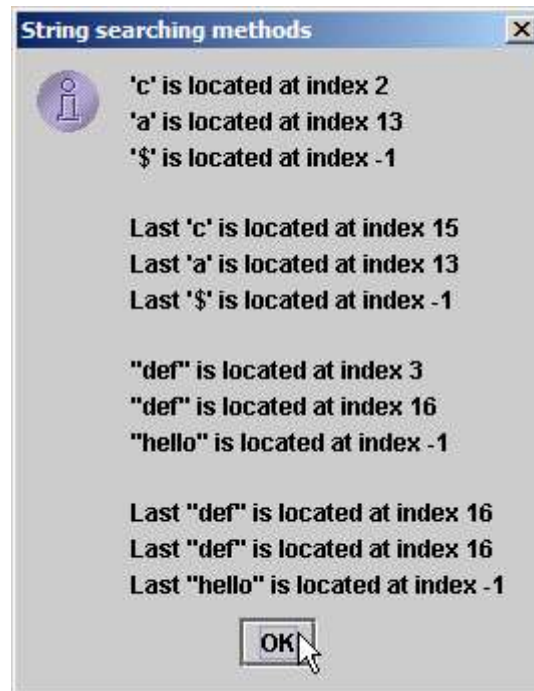
```



Outline



StringIndexMeth
ods.java



11.3.5 Extracting Substrings from Strings

- Create Strings from other Strings
 - Method substring
 - substring(start) (all the way to the end)
 - substring(start, first after)





Outline

SubString.java

Line 13

Line 16

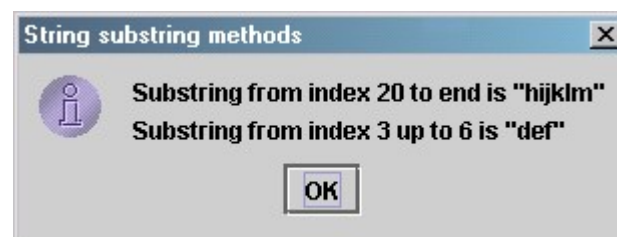
```

1  // Fig. 11.6: SubString.java
2  // String class substring methods.
3  import javax.swing.*;
4
5  public class SubString {
6
7      public static void main( String args[] )
8      {
9          String letters = "abcdefghijklm";
10
11         // test substring methods
12         String output = "Substring from index 20 to end is " +
13             "\"\" + letters.substring( 20 ) + "\"\n";
14
15         output += "Substring from index 3 up to 6 is " +
16             "\"\" + letters.substring( 3, 6 ) + "\"";
17
18         JOptionPane.showMessageDialog( null, output,
19             "String substring methods", JOptionPane.INFORMATION_MESSAGE );
20
21         System.exit( 0 );
22     }
23
24 } // end class SubString

```

Beginning at index 20,
extract characters from
String letters

Extract characters from index 3
to 6 from String letters



11.3.6 Concatenating Strings

- Method `concat`
 - Concatenate two `String` objects
 - `s1.concat( s2 )`



StringConcatena
tion.java

Line 14

```
1  // Fig. 11.7: StringConcatenation.java
2  // String concat method.
3  import javax.swing.*;
4
5  public class StringConcatenation {
6
7      public static void main( String args[] )
8      {
9          String s1 = new String( "Happy " );
10         String s2 = new String( "Birthday" );
11
12         String output = "s1 = " + s1 + "\ns2 = " + s2;
13
14         output += "\n\nResult of s1.concat( s2 ) = " + s1.concat( s2 );
15         output += "\ns1 after concatenation = " + s1;
16
17         JOptionPane.showMessageDialog( null, output,
18             "String method concat", JOptionPane.INFORMATION_MESSAGE );
19
20         System.exit( 0 );
21     }
22
23 } // end class StringConcatenation
```

Concatenate String s2
to String s1

However, String s1 is not
modified by method concat



11.3.7 Miscellaneous String Methods

- Miscellaneous `String` methods
 - Return modified copies of `String`
 - `replace(char, char)`
 - `toUpperCase()`
 - `toLowerCase()`
 - `trim()` (remove all white space from beginning and end of string)
 - Return character array
 - `toArray()`





Outline

StringMiscellaneous2.java

```

1  // Fig. 11.8: StringMiscellaneous2.java
2  // String methods replace, toLowerCase, toUpperCase, trim and toCharArray.
3  import javax.swing.*;
4
5  public class StringMiscellaneous2 {
6
7      public static void main( String args[] )
8      {
9          String s1 = new String( "hello" );
10         String s2 = new String( "GOODBYE" );
11         String s3 = new String( "  spaces  " );
12
13         String output = "s1 = " + s1 + "\ns2 = " + s2 + "\ns3 = ";
14
15         // test method replace
16         output += "\n\nReplace 'l' with 'L' in s1: " +
17             s1.replace( 'l', 'L' );
18
19         // test toLowerCase and toUpperCase
20         output += "\n\ns1.toUpperCase() = " + s1.toUpperCase()
21             "\ns2.toLowerCase() = " + s2.toLowerCase();
22
23         // test trim method
24         output += "\n\ns3 after trim = \"" + s3.trim() + "\"";
25

```

Use method **replace** to return **s1** copy in which every occurrence of 'l' is replaced with 'L'

Use method **toUpperCase** to return **s1** copy in which every character is uppercase

Use method **toLowerCase** to return **s2** copy in which every character is lowercase

Use method **trim** to return **s3** copy in which whitespace is eliminated

Outline

StringMiscellaneous2.java

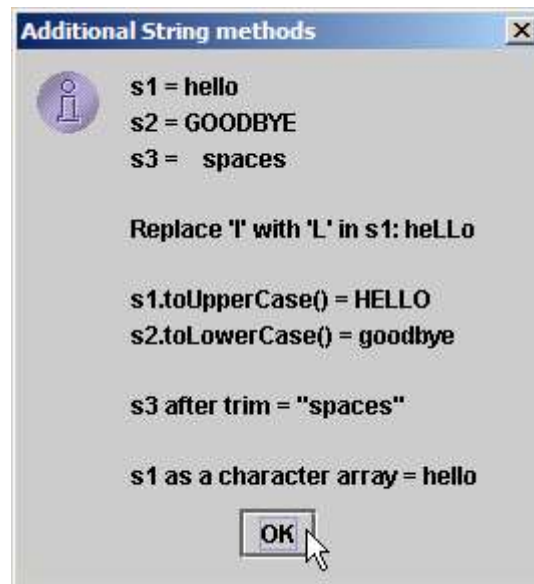
Line 27

```

26 // test toCharArray method
27 char charArray[] = s1.toCharArray(); ←
28 output += "\n\ns1 as a character array = ";
29
30 for ( int count = 0; count < charArray.length; ++count )
31     output += charArray[ count ];
32
33 JOptionPane.showMessageDialog( null, output,
34     "Additional String methods", JOptionPane.INFORMATION_MESSAGE );
35
36 system.exit( 0 );
37 }
38
39 } // end class StringMiscellaneous2

```

Use method toCharArray to
return character array of s1



11.3.8 String Method `valueOf`

- `String` provides `static` class methods
 - Method `valueOf`
 - Returns `String` representation of object, data, etc.
 - `toString` cannot be used with primitives, but `valueOf` can





Outline



StringValueOf.j
ava

Lines 20-26

```
1 // Fig. 11.9: StringValueOf.java
2 // String valueOf methods.
3 import javax.swing.*;
4
5 public class StringValueOf {
6
7     public static void main( String args[] )
8     {
9         char charArray[] = { 'a', 'b', 'c', 'd', 'e', 'f' };
10        boolean booleanValue = true;
11        char characterValue = 'Z';
12        int integerValue = 7;
13        long longValue = 10000000L;
14        float floatValue = 2.5f; // f suffix indicates that 2.5 is a float
15        double doubleValue = 33.333;
16        Object objectRef = "hello"; // assign string to an Object reference
17
18        String output = "char array = " + String.valueOf( charArray ) +
19            "\npart of char array = " + String.valueOf( charArray, 3, 3 ) +
20            "\nboolean = " + String.valueOf( booleanValue ) +
21            "\nchar = " + String.valueOf( characterValue ) +
22            "\nint = " + String.valueOf( integerValue ) +
23            "\nlong = " + String.valueOf( longValue ) +
24            "\nfloat = " + String.valueOf( floatValue ) +
25            "\ndouble = " + String.valueOf( doubleValue ) +
26            "\nObject = " + String.valueOf( objectRef );
```

static method valueOf of
class **String** returns **String**
representation of various types

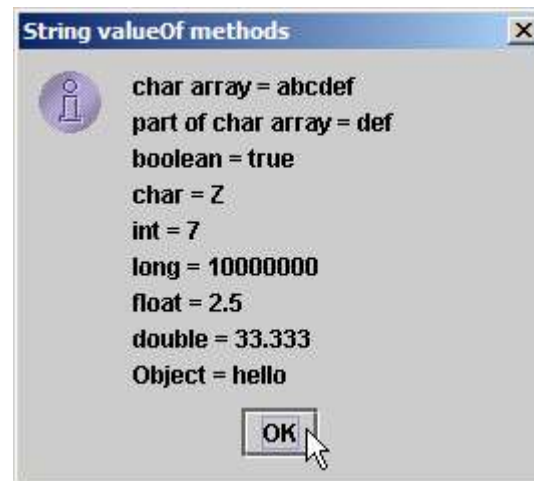


Outline



StringValueOf.j
ava

```
27
28     JOptionPane.showMessageDialog( null, output,
29         "String valueOf methods", JOptionPane.INFORMATION_MESSAGE );
30
31     system.exit( 0 );
32 }
33
34 } // end class StringValueOf
```



11.4 Class StringBuffer

- Class `StringBuffer`
 - When `String` object is created, its contents cannot change
 - `StringBuffer` used for creating and manipulating dynamic string data
 - i.e., modifiable `Strings`
 - Can store characters based on capacity
 - Capacity expands dynamically to handle additional characters
 - Uses operators `+` and `+=` for `String` concatenation



11.4.1 StringBuffer Constructors

- Three `StringBuffer` constructors
 - Default creates `StringBuffer` with no characters
 - Capacity of 16 characters
 - `toString` method can be used to convert `StringBuffer` object into `String` object





```

1  // Fig. 11.10: StringBufferConstructors.java
2  // StringBuffer constructors.
3  import javax.swing.*;
4
5  public class StringBufferConstructors {
6
7      public static void main( String args[] )
8      {
9          StringBuffer buffer1 = new StringBuffer();
10         StringBuffer buffer2 = new StringBuffer( 10 );
11         StringBuffer buffer3 = new StringBuffer( "hello" );
12
13         String output = "buffer1 = \"\" + buffer1.toString() + "\"\" +
14             "\nbuffer2 = \"\" + buffer2.toString() + "\"\" +
15             "\nbuffer3 = \"\" + buffer3.toString() + "\"\"";
16
17         JOptionPane.showMessageDialog( null, output,
18             "StringBuffer constructors", JOptionPane.INFORMATION_MESSAGE );
19
20         system.exit( 0 );
21     }
22
23 } // end class StringBufferConstructors

```

Default constructor creates empty **StringBuffer** with capacity of **16** characters

Second constructor creates empty **StringBuffer** with capacity of specified (**10**) characters

Third constructor creates **StringBuffer** with **String "hello"** and capacity of **21** characters

Method **toString** returns **String** representation of **StringBuffer**



11.4.2 StringBuffer Methods `length`, `capacity`, `setLength` and `ensureCapacity`

- Method `length`
 - Return `StringBuffer` length
- Method `capacity`
 - In general, capacity is 16 characters more than initial string
 - Return `StringBuffer` capacity
- Method `setLength`
 - Increase or decrease `StringBuffer` length
 - Characters may be discarded or null characters added
- Method `ensureCapacity`
 - Set `StringBuffer` capacity
 - Guarantee that `StringBuffer` has minimum capacity





Outline



StringBufferCap
Len.java

```

1  // Fig. 11.11: StringBufferCapLen.java
2  // StringBuffer length, setLength, capacity and ensureCapacity methods.
3  import javax.swing.*;
4
5  public class StringBufferCapLen {
6
7      public static void main( String args[] )
8      {
9          StringBuffer buffer = new StringBuffer( "Hello, how are you?" );
10
11         String output = "buffer = " + buffer.toString() + "\nlength = " +
12             buffer.length() + "\ncapacity = " + buffer.capacity();
13
14         buffer.ensureCapacity( 75 );
15         output += "\n\nNew capacity = " + buffer.capacity();
16
17         buffer.setLength( 10 );
18         output += "\n\nNew length = " + buffer.length() +
19             "\nbuf = " + buffer.toString();
20
21         JOptionPane.showMessageDialog( null, output,
22             "StringBuffer length and capacity Methods",
23             JOptionPane.INFORMATION_MESSAGE );
24     }

```

Method length returns
StringBuffer length

Line 12

Method capacity returns
StringBuffer capacity

Use method ensureCapacity
to set capacity to 75

Use method setLength
to set length to 10



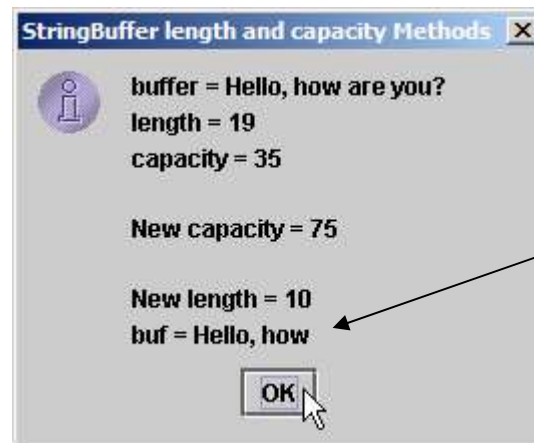
Outline



```
25     System.exit( 0 );  
26 }  
27  
28 } // end class StringBufferCapLen
```

StringBufferCap
Len.java

Only 10 characters
from
StringBuffer are
printed



Only 10 characters from
StringBuffer are printed

11.4.3 StringBuffer Methods charAt, setCharAt, getChars and reverse

- Manipulating StringBuffer characters
 - Method charAt
 - Return StringBuffer character at specified index
 - Method setCharAt
 - Set StringBuffer character at specified index
 - Method getChars
 - Return character array from StringBuffer
 - getChars (start, first after, char array, start)
 - Method reverse
 - Reverse StringBuffer contents





Outline



StringBufferChars.java

```

1  // Fig. 11.12: StringBufferChars.java
2  // StringBuffer methods charAt, setCharAt, getChars and reverse.
3  import javax.swing.*;
4
5  public class StringBufferChars {
6
7      public static void main( String args[] )
8      {
9          StringBuffer buffer = new StringBuffer( "hello there" );
10
11          String output = "buffer = " + buffer.toString() +
12                        "\ncharacter at 0: " + buffer.charAt( 0 ) +
13                        "\ncharacter at 4: " + buffer.charAt( 4 );
14
15          char charArray[] = new char[ buffer.length() ];
16          buffer.getChars( 0, buffer.length(), charArray, 0 );
17          output += "\n\nThe characters are: ";
18
19          for ( int count = 0; count < charArray.length; ++count )
20              output += charArray[ count ];
21
22          buffer.setCharAt( 0, 'H' );
23          buffer.setCharAt( 6, 'T' );
24          output += "\n\nbuf = " + buffer.toString();
25

```

Return StringBuffer
characters at indices 0
and 4, respectively

Lines 22-23

Return character array
from StringBuffer

Replace characters at
indices 0 and 6 with 'H'
and 'T,' respectively

Outline

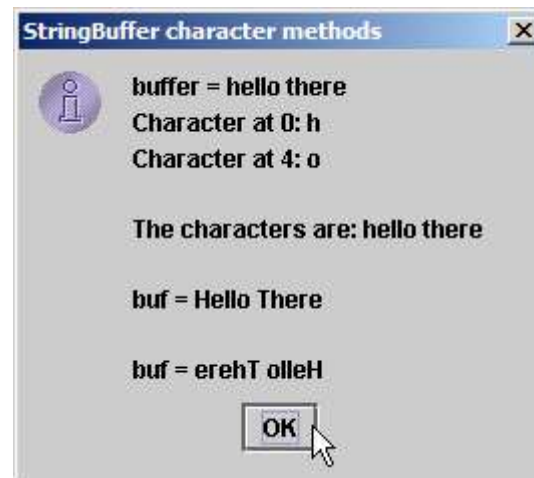
StringBufferChars.java

Lines 26



Reverse characters in
StringBuffer

```
26     buffer.reverse();
27     output += "\n\nbuf = " + buffer.toString();
28
29     JOptionPane.showMessageDialog( null, output,
30         "StringBuffer character methods",
31         JOptionPane.INFORMATION_MESSAGE );
32
33     system.exit( 0 );
34 }
35
36 } // end class StringBufferChars
```



11.4.4 StringBuffer append Methods

- Method **append**
 - Allow data values to be added to the end of a `StringBuffer` object
 - `string1 + string2` compiled as `StringBuffer(string1).append(string2)`
 - `string1 += string2` compiled as `string1 = StringBuffer(string1).append(string2)`





Outline

StringBufferApp
end.java

Line 21

Line 23

Line 25

Line 27

```

1  // Fig. 11.13: StringBufferAppend.java
2  // StringBuffer append methods.
3  import javax.swing.*;
4
5  public class StringBufferAppend {
6
7      public static void main( String args[] )
8      {
9          Object objectRef = "hello";
10         String string = "goodbye";
11         char charArray[] = { 'a', 'b', 'c', 'd', 'e', 'f' };
12         boolean booleanValue = true;
13         char characterValue = 'Z';
14         int integerValue = 7;
15         long longValue = 10000000;
16         float floatValue = 2.5f; // f suffix indicates 2.5 is a float
17         double doubleValue = 33.333;
18         StringBuffer lastBuffer = new StringBuffer( "last StringBuffer" );
19         StringBuffer buffer = new StringBuffer();
20
21         buffer.append( objectRef );
22         buffer.append( " " ); // each of these contains two
23         buffer.append( string );
24         buffer.append( " " );
25         buffer.append( charArray );
26         buffer.append( " " );
27         buffer.append( charArray, 0, 3 );

```

Append String "hello"
to StringBuffer

Append String "goodbye"

Append "a b c d e f"

Append "a b c"



Outline

StringBufferApp

Append boolean, char, int, a
long, float and double

Line 29-39

```

28     buffer.append( " " );
29     buffer.append( booleanValue );
30     buffer.append( " " );
31     buffer.append( characterValue );
32     buffer.append( " " );
33     buffer.append( integerValue );
34     buffer.append( " " );
35     buffer.append( longValue );
36     buffer.append( " " );
37     buffer.append( floatValue );
38     buffer.append( " " );
39     buffer.append( doubleValue );
40     buffer.append( " " );
41     buffer.append( lastBuffer );
42
43     JOptionPane.showMessageDialog( null,
44         "buffer = " + buffer.toString(), "StringBuffer append Methods",
45         JOptionPane.INFORMATION_MESSAGE );
46
47     System.exit( 0 );
48 }
49
50 } // end StringBufferAppend

```



11.4.5 StringBuffer Insertion and Deletion Methods

- Method `insert`
 - Allow data-type values to be inserted into `StringBuffer`
 - `insert (before, object)`
- Methods `delete` and `deleteCharAt`
 - Allow characters to be removed from `StringBuffer`
 - `delete (start, first after)`
 - `deleteCharAt (index)`





Outline

StringBufferInsert.java

Lines 20-26

```
1 // Fig. 11.14: StringBufferInsert.java
2 // StringBuffer methods insert and delete.
3 import javax.swing.*;
4
5 public class StringBufferInsert {
6
7     public static void main( String args[] )
8     {
9         Object objectRef = "hello";
10        String string = "goodbye";
11        char charArray[] = { 'a', 'b', 'c', 'd', 'e', 'f' };
12        boolean booleanValue = true;
13        char characterValue = 'K';
14        int integerValue = 7;
15        long longValue = 10000000;
16        float floatValue = 2.5f; // f suffix indicates that 2.5 is a float
17        double doubleValue = 33.333;
18        StringBuffer buffer = new StringBuffer();
19
20        buffer.insert( 0, objectRef );
21        buffer.insert( 0, "  " ); // each of these contains two spaces
22        buffer.insert( 0, string );
23        buffer.insert( 0, "  " );
24        buffer.insert( 0, charArray );
25        buffer.insert( 0, "  " );
26        buffer.insert( 0, charArray, 3, 3 );
```

Use method **insert** to insert data in beginning of **StringBuffer**

Outline

```

27     buffer.insert( 0, " " );
28     buffer.insert( 0, booleanValue );
29     buffer.insert( 0, " " );
30     buffer.insert( 0, characterValue );
31     buffer.insert( 0, " " );
32     buffer.insert( 0, integerValue );
33     buffer.insert( 0, " " );
34     buffer.insert( 0, longValue );
35     buffer.insert( 0, " " );
36     buffer.insert( 0, floatValue );
37     buffer.insert( 0, " " );
38     buffer.insert( 0, doubleValue );

```

Use method `insert` to insert
data in beginning of
`StringBuffer`

`StringBufferInsert.java`

Lines 27-38

Line 42

Use method `deleteCharAt` to
remove character from index 10 in
`StringBuffer`

```

40     String output = "buffer after inserts:\n" + buffer.toString();
41
42     buffer.deleteCharAt( 10 ); // delete 5 in 2.5
43     buffer.delete( 2, 6 );    // delete .333 in 33.333
44
45     output += "\n\nbuffer after deletes:\n" + buffer.toString();
46
47     JOptionPane.showMessageDialog( null, output,
48         "StringBuffer insert/delete", JOptionPane.INFORMATION_MESSAGE );
49
50     System.exit( 0 );
51 }
52
53 } // end class StringBufferInsert

```

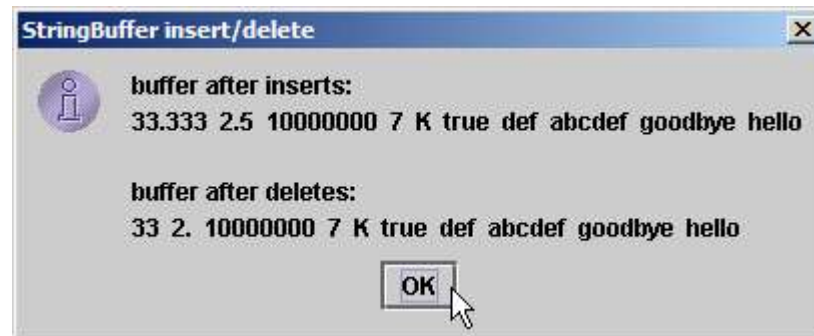
Remove characters from
indices 2 through 5 (inclusive)



Outline

46

StringBufferInsert.java



11.5 Class Character

- Treat primitive variables as objects
 - Type wrapper classes
 - Boolean
 - Character
 - Double
 - Float
 - Byte
 - Short
 - Integer
 - Long
 - We examine class Character





Outline



StaticCharMetho
ds.java

```
1 // Fig. 11.15: StaticCharMethods.java
2 // Static Character testing methods and case conversion methods.
3 import java.awt.*;
4 import java.awt.event.*;
5 import javax.swing.*;
6
7 public class StaticCharMethods extends JFrame {
8     private char c;
9     private JLabel promptLabel;
10    private JTextField inputField;
11    private JTextArea outputArea;
12
13    // constructor builds GUI
14    public StaticCharMethods()
15    {
16        super( "Static Character Methods" );
17
18        Container container = getContentPane();
19        container.setLayout( new FlowLayout() );
20
21        promptLabel = new JLabel( "Enter a character and press Enter" );
22        container.add( promptLabel );
23        inputField = new JTextField( 5 );
24    }
```




Outline



StaticCharMetho ds.java

```
25     inputField.addActionListener(  
26  
27         new ActionListener() { // anonymous inner class  
28  
29             // handle textfield event  
30             public void actionPerformed((ActionEvent event) )  
31             {  
32                 String s = event.getActionCommand();  
33                 c = s.charAt( 0 );  
34                 buildOutput();  
35             }  
36  
37         } // end anonymous inner class  
38  
39     ); // end call to addActionListener  
40  
41     container.add( inputField );  
42     outputArea = new JTextArea( 10, 20 );  
43     container.add( outputArea );  
44  
45     setSize( 300, 220 ); // set the window size  
46     setVisible( true ); // show the window  
47  
48 } // end constructor  
49
```



```

50 // display character info in outputArea
51 private void buildOutput()
52 {
53     outputArea.setText( "is defined: " + Character.isDefine
54         "\nis digit: " + Character.isDigit( c ) +
55         "\nis first character in a Java identifier: " +
56         Character.isJavaIdentifierStart( c ) +
57         "\nis part of a Java identifier: " +
58         Character.isJavaIdentifierPart( c ) +
59         "\nis letter: " + Character.isLetter( c ) +
60         "\nis letter or digit: " + Character.isLetterOrDigit( c )
61         "\nis lower case: " + Character.isLowerCase( c ) +
62         "\nis upper case: " + Character.isUpperCase( c ) +
63         "\nto upper case: " + Character.toUpperCase( c ) +
64         "\nto lower case: " + Character.toLowerCase( c ) );
65 }
66
67 // create StaticCharMethods object to begin execution
68 public static void main( String args[] )
69 {
70     StaticCharMethods application = new StaticCharMethods();
71     application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
72 }
73
74 } // end class StaticCharMethods

```

Determine whether **c** is defined Unicode digit

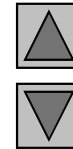
Determine whether **c** can be used as first character in identifier

Determine whether **c** can be used as identifier character

Determine whether **c** is a letter

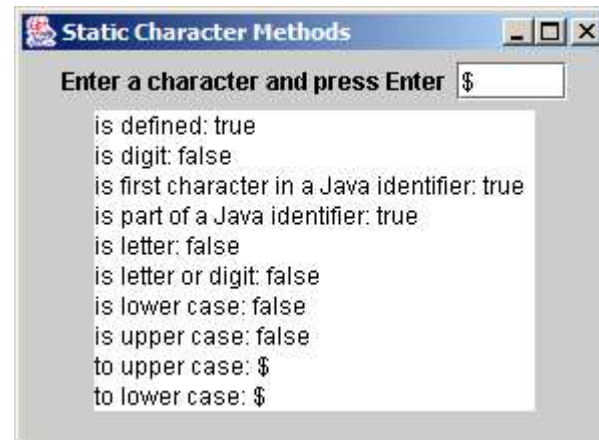
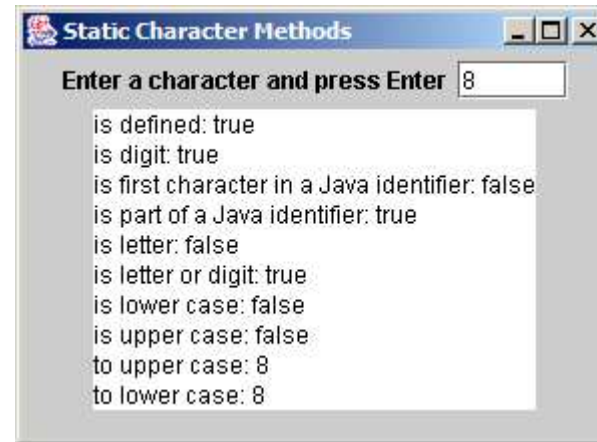
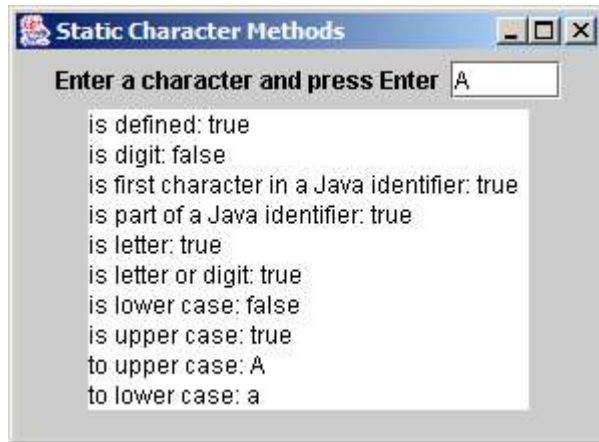
Determine whether **c** is letter or digit

Determine whether **c** is uppercase or lowercase



Outline

StaticCharMethods.java



11.6 Class StringTokenizer

- `java.util.StringTokenizer`
 - Partition `String` into individual tokens (substrings, words)
 - Use *delimiter* (default is space, tab, newline, return)
 - `StringTokenizer tokens = new StringTokenizer(addressLine);`
 - `StringTokenizer tokens = new StringTokenizer(addressLine, “ ,;.?!”);`
 - `tokens.countTokens()` (number of tokens in the string)
 - `tokens.nextToken()` (return next token)
 - `tokens.hasMoreTokens()` (true or false)



Outline

TokenTest.java

Line 24

```
1  // Fig. 11.18: TokenTest.java
2  // StringTokenizer class.
3  import java.util.*;
4  import java.awt.*;
5  import java.awt.event.*;
6  import javax.swing.*;
7
8  public class TokenTest extends JFrame {
9      private JLabel promptLabel;
10     private JTextField inputField;
11     private JTextArea outputArea;
12
13     // set up GUI and event handling
14     public TokenTest()
15     {
16         super( "Testing Class StringTokenizer" );
17
18         Container container = getContentPane();
19         container.setLayout( new FlowLayout() );
20
21         promptLabel = new JLabel( "Enter a sentence and press Enter" );
22         container.add( promptLabel );
23
24         inputField = new JTextField( 20 );
```

inputField contains String to
be parsed by StringTokenizer



```

25  inputField.addActionListener(
26
27      new ActionListener() { // anonymous inner class
28
29          // handle text field event
30          public void actionPerformed( ActionEvent event )
31          {
32              StringTokenizer tokens =
33                  new StringTokenizer( event.getActionCommand() );
34
35              outputArea.setText( "Number of elements: " +
36                  tokens.countTokens() + "\nThe tokens are:\n" );
37
38              while ( tokens.hasMoreTokens() )
39                  outputArea.append( tokens.nextToken() + "\n" );
40          }
41
42      } // end anonymous inner class
43
44  ); // end call to addActionListener
45
46  container.add( inputField );
47
48  outputArea = new JTextArea( 10, 20 );
49  outputArea.setEditable( false );
50  container.add( new JScrollPane( outputArea ) );
51  setSize( 275, 240 ); // set the window size
52  setVisible( true ); // show the window
53  }

```

Use **StringTokenizer** to parse **String**
using default delimiter " \n\t\r"

Line 33

Count number of tokens

Lines 38-39

Append next token to **outputArea**,
as long as tokens exist



Outline



TokenTest.java

```
54
55 // execute application
56 public static void main( String args[] )
57 {
58     TokenTest application = new TokenTest();
59     application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
60 }
61
62 } // end class TokenTest
```

